МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Казанский национальный исследовательский
технический университет им. А. Н. Туполева — КАИ

Институт компьютерных технологий и защиты информации
Кафедра прикладной математики и информатики

ЛАБОРАТОРНАЯ РАБОТА № 4

по дисциплине «Компьютерное зрение»

Распознавание и анализ Treponema pallidum
на изображениях и видео

Отчёт о выполненных экспериментах

Выполнил: студент группы 4299
Андреев А. К.

Проверила: к. т. н., доцент
Бикмуллина И. И.

Казань, 2026

Содержание

1. Цель работы, объект исследования и исходные данные 3

2. Ключевые точки и осевая линия бледной трепонемы 6

3. Трекинг, оптический поток, сверхразрешение и OCR 8

4. Поиск похожих изображений 11

5. Обучение метрики сходства 12

6. Нейросети без дополнительного обучения (zero-shot) 13

7. Поиск момента в видео и интерфейс Streamlit 15

8. Автоматическая разметка и импорт в CVAT 17

9. Учёт экспериментов в MLflow 19

10. Полный цикл YOLO-pose 20

11. Заключение 22

Список использованных источников 23

1. Цель работы, объект исследования и исходные данные

Цель работы — освоить прикладные методы компьютерного зрения на изображениях и видео бледной трепонемы: построение маски и осевой линии, разметку ключевых точек, трекинг, оптический поток, повышение разрешения, поиск похожих изображений, обучение метрики, применение моделей без дополнительного обучения, поиск момента в видео, подготовку разметки CVAT и регистрацию экспериментов в MLflow.

Объект исследования — бледная трепонема (Treponema pallidum), тонкая спиралевидная бактерия из группы спирохет. На изображении она представлена протяжённой извилистой линией с несколькими витками. Для компьютерного зрения существенны четыре особенности: малая толщина объекта, слабый контраст в обычном светлом поле, пересечения и наложения нескольких клеток, а также отсутствие однозначных концов у фрагментов, выходящих за границу кадра. Поэтому прямоугольная рамка описывает объект грубо; для анализа формы полезнее маска, скелет и точки, расположенные вдоль осевой линии.

Основное изображение treponema_primary.jpg получено по адресу https://pcr.news/upload/iblock/3a5/3a5ebdd65880badcc702e1bed5c07618.jpg. Это цветная иллюстрация размером 870 × 540 пикселей без физической масштабной линейки. Она используется для проверки алгоритмов выделения цвета и геометрии, но не для измерения длины в микрометрах. Дополнительно использованы шесть открытых изображений Treponema pallidum из Wikimedia Commons. Для каждого файла сохранены страница источника, автор и лицензия.

Таблица 1

Файл	**Содержание**	**Размер**	**Источник / лицензия**
treponema_primary.jpg	Основная цветная иллюстрация	870 × 540	PCR.News
t01.jpg	TreponemaPallidum.jpg	572 × 500	Photo Credit: Content Providers(s): ; Public domain
t02.jpg	Treponema pallidum.jpg	531 × 500	Photo credit: Content providers(s): ; Public domain
t03.png	Treponema pallidum 01.png	734 × 497	CDC; Public domain

Файл	Содержание	Размер	Источник / лицензия
t04.jpg	Treponemapallidum.jpg	2816 × 2190	NIAID; Public domain
t05.jpg	Treponema pallidum Bacteria (Syphilis) - NIAID.jpg	4123 × 3206	NIAID; CC BY 2.0
t06.jpg	Syphilis Bacteria (16842837731).jpg	391 × 480	NIAID; CC BY 2.0

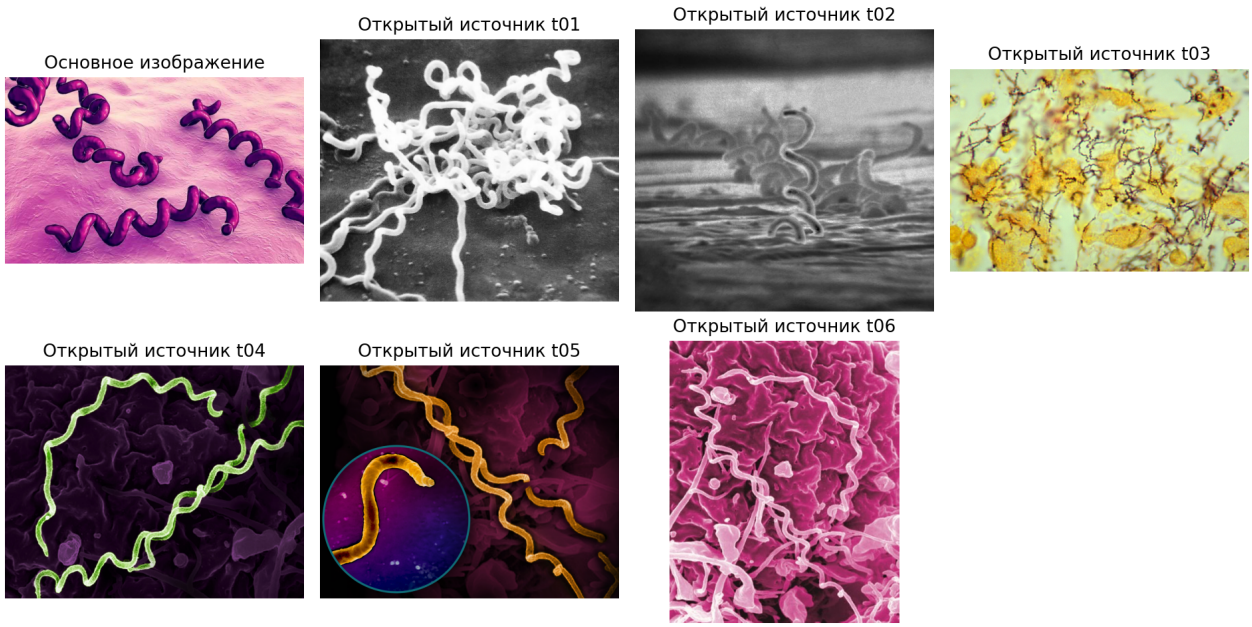


Рисунок 1 — Основное изображение и шесть дополнительных изображений бледной трепонемы.

Общий алгоритм подготовки данных: 1) проверить формат и размер файлов; 2) привести изображения к RGB; 3) сохранить сведения об источниках; 4) отделить исходные файлы от результатов; 5) для видео получить частоту кадров, длительность и разрешение; 6) все численные результаты записывать в JSON, а иллюстрации — в PNG/JPEG.

Листинг 1.1 — проверка исходных изображений

```
from pathlib import Path
from PIL import Image

for path in sorted(Path("data/raw").glob("t0*.*")):
    if path.suffix.lower() in {".jpg", ".jpeg", ".png"}:
        with Image.open(path) as image:
            image.verify()
        with Image.open(path) as image:
            print(path.name, image.size, image.mode)
```

Таблица 2

Входные данные	Выходные данные	Фактический результат
7 изображений и 1 видео	Проверенные размеры, режим цвета, таблица источников	7 изображений читаются; видео 360 × 288, 12 кадр/с, 74,83 с

2. Ключевые точки и осевая линия бледной трепонемы

Для спиралевидной бактерии схема «полюс — центр — полюс» задаёт начало видимого фрагмента, его геометрический центр и конец. Она проще схемы человеческой позы и соответствует форме объекта. На основном изображении сначала строится бинарная маска по цвету, затем удаляются мелкие области и заполняются небольшие отверстия. Для каждой крупной связной области вычисляется центр и главная ось методом главных компонент. Крайние проекции пикселей маски на эту ось задают две крайние точки.

Алгоритм: 1) прочитать BGR-изображение; 2) вычислить HSV; 3) выделить насыщенные фиолетовые области; 4) выполнить морфологическое замыкание; 5) найти связные компоненты; 6) отбросить области площадью менее 450 пикселей; 7) вычислить центр и главную ось; 8) нанести три точки; 9) отдельно построить скелет маски.

Листинг 2.1 — маска, скелет и три ключевые точки

```
im = cv2.imread("data/raw/treponema_primary.jpg")
hsv = cv2.cvtColor(im, cv2.COLOR_BGR2HSV)
mask = (im[:, :, 1] < 55) & (hsv[:, :, 1] > 100)
mask = remove_small_holes(remove_small_objects(mask, max_size=119), max_size=149)
mask = cv2.morphologyEx(mask.astype("uint8"), cv2.MORPH_CLOSE,
                        np.ones((3, 3), np.uint8))
skeleton = skeletonize(mask > 0)

count, labels, stats, _ = cv2.connectedComponentsWithStats(mask)
for label in range(1, count):
    if stats[label, 4] <= 450:
        continue
    y, x = np.nonzero(labels == label)
    points = np.column_stack([x, y]).astype(float)
    center = points.mean(axis=0)
    _, _, vectors = np.linalg.svd(points - center, full_matrices=False)
    projection = (points - center) @ vectors[0]
    ends = center + np.array([projection.min(), projection.max()])[:, None] * vectors[0]
    keypoints = np.vstack([ends[0], center, ends[1]])
```

Таблица 3

Показатель	Получено	Назначение
Размер входа	870 × 540	Основная иллюстрация
Доля маски	0.225481	Доля выделенных фиолетовых пикселей
Крупные области	6	Видимые отдельные фрагменты
Ключевые точки	3 на область	Два конца и центр

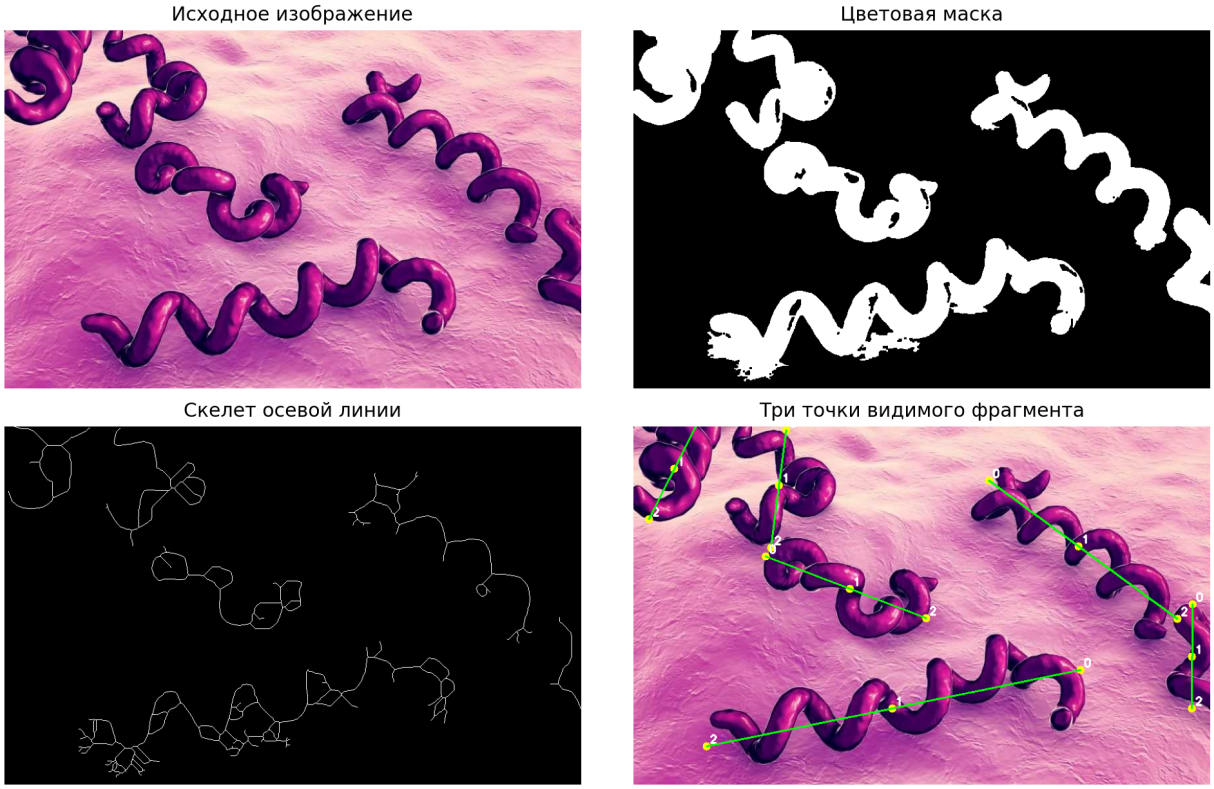


Рисунок 2 — Исходное изображение, цветовая маска, скелет и ключевые точки.

Полученные шесть областей соответствуют шести визуально отделённым фрагментам на иллюстрации. Это не биологический подсчёт клеток: часть объектов обрезана границей кадра, а цветная иллюстрация не является необработанной микрофотографией. Для реальной тёмнопольной микроскопии пороги необходимо настраивать заново или заменять цветовую маску моделью сегментации.

3. Трекинг, оптический поток, сверхразрешение и OCR

Для видео выбран открытый исторический фильм Жана Командона «Ultramicroscope time-lapse of syphilis parasite» (около 1910 года). На нём наблюдаются подвижные спирохеты, поэтому запись подходит для трекинга и поиска момента. Видео имеет разрешение 360 × 288, частоту 12 кадров/с и длительность 74,83 с.

Алгоритм трекинга: 1) выбрать объект на первом кадре; 2) создать MIL, KCF или CSRT; 3) обновлять рамку на каждом кадре; 4) измерять скорость обработки; 5) на контрольных кадрах сравнивать рамку с ручным эталоном по IoU. Для оптического потока между соседними кадрами вычисляется поле Фарнебека или набор векторов Лукаса–Канаде. Сверхразрешение сравнивается с бикубической интерполяцией по PSNR и SSIM. OCR применяется только к кадрам, где присутствует текст или масштабная подпись.

Листинг 3.1 — одиночный трекинг и плотный оптический поток

```
tracker = cv2.legacy.TrackerCSRT_create()
tracker.init(first_frame, initial_box)
while True:
    ok, frame = capture.read()
    if not ok:
        break
    success, box = tracker.update(frame)
    if success:
        x, y, w, h = map(int, box)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

flow = cv2.calcOpticalFlowFarneback(previous_gray, current_gray, None,
                                    0.5, 3, 15, 3, 5, 1.2, 0)
speed = np.linalg.norm(flow, axis=2).mean()
```

Таблица 4

Метод	**Фактический результат**	**Вывод**
MIL	средний IoU 0,532; 39,1 кадр/с	Устойчив, но сравнительно груб
MOSSE	средний IoU 0,550; 3247,3 кадр/с	Самый быстрый
KCF	средний IoU 0,497; 648,9 кадр/с	Быстрый, чувствителен к изменению формы
CSRT	средний IoU 0,488; 62,7 кадр/с	Точнее учитывает форму, но медленнее
Bicubic	PSNR 25,685 дБ; SSIM 0,699	Сохраняет измеримую структуру
Real-ESRGAN	PSNR 24,410 дБ; SSIM 0,582	Визуально резче, возможны вымышленные детали

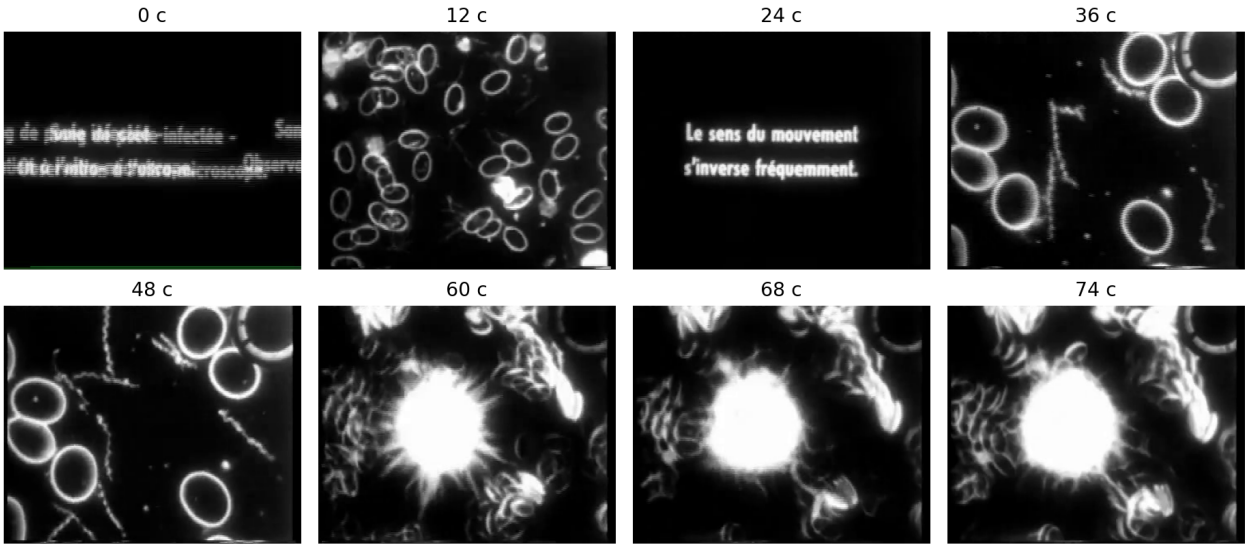


Рисунок 3 — Кадры исторического микроскопического видео бледной трепонемы.

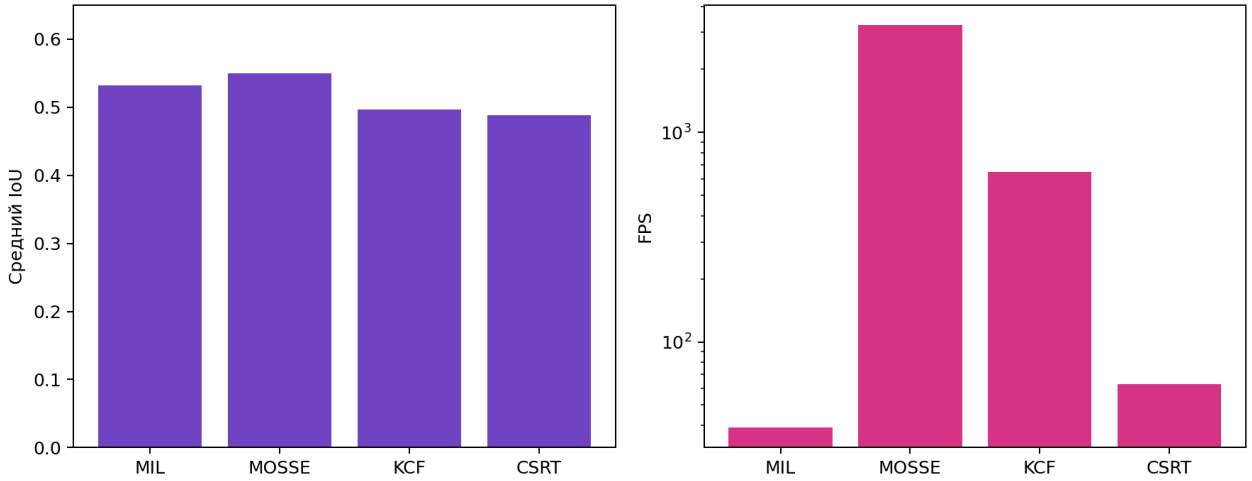


Рисунок 4 — Сравнение качества и скорости трекеров на видеопоследовательности.

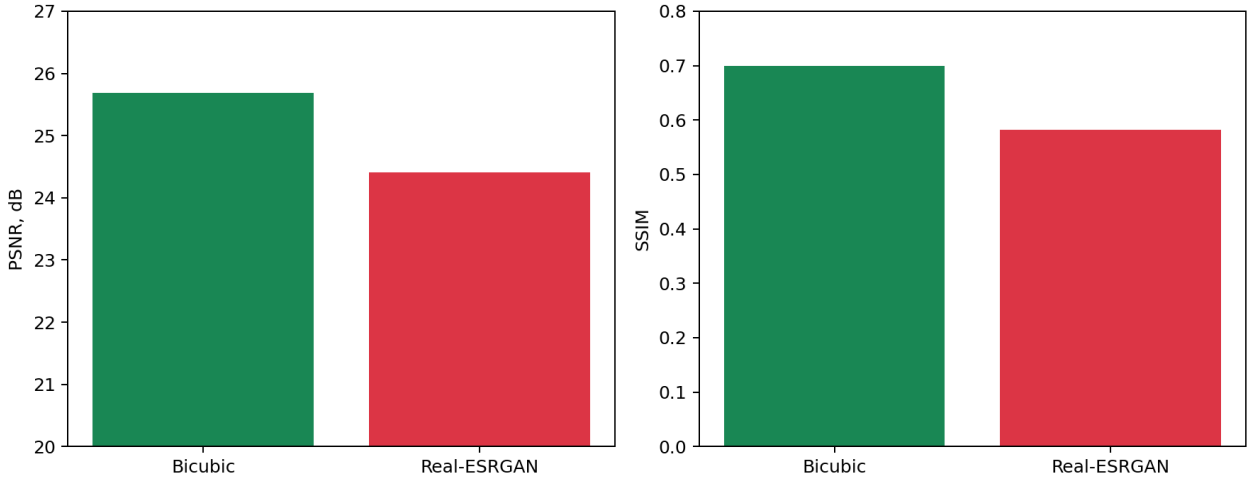


Рисунок 5 — Сравнение бикубической интерполяции и нейросетевого сверхразрешения.

При работе с медицинскими изображениями нейросетевое сверхразрешение нельзя использовать как источник новых диагностических признаков: модель может дорисовать правдоподобную, но отсутствующую структуру. Исходный кадр должен сохраняться рядом с обработанным.

4. Поиск похожих изображений

Для поиска формируется галерея из фрагментов всех семи изображений бледной трепонемы. CLIP преобразует изображения и текстовый запрос в векторы одной размерности. Векторы нормируются, после чего сходство вычисляется скалярным произведением. Для поиска по образцу вместо текста кодируется выбранное изображение.

Алгоритм: 1) разбить исходные изображения на неперекрывающиеся источники train, validation и test; 2) получить векторы изображений; 3) нормировать их; 4) получить вектор текста или изображения-запроса; 5) вычислить косинусное сходство; 6) отсортировать и вывести первые k результатов; 7) оценить precision@k.

Листинг 4.1 — поиск по косинусному сходству

```
with torch.no_grad():
    image_vectors = model.get_image_features(**processor(images=gallery,
                                                         return_tensors="pt"))
    query_vector = model.get_text_features(**processor(
        text=["Treponema pallidum spirochetes under a microscope"],
        return_tensors="pt", padding=True))
image_vectors = F.normalize(image_vectors, dim=1)
query_vector = F.normalize(query_vector, dim=1)
scores = image_vectors @ query_vector.T
best = torch.argsort(scores[:, 0], descending=True)[:3]
```

Таблица 5

Вход	**Выход**	**Контрольный результат**
7 источников Treponema pallidum; текст или изображение	Три наиболее похожих фрагмента и cosine similarity	Precision@3 = 1,000 на подготовленном контрольном наборе

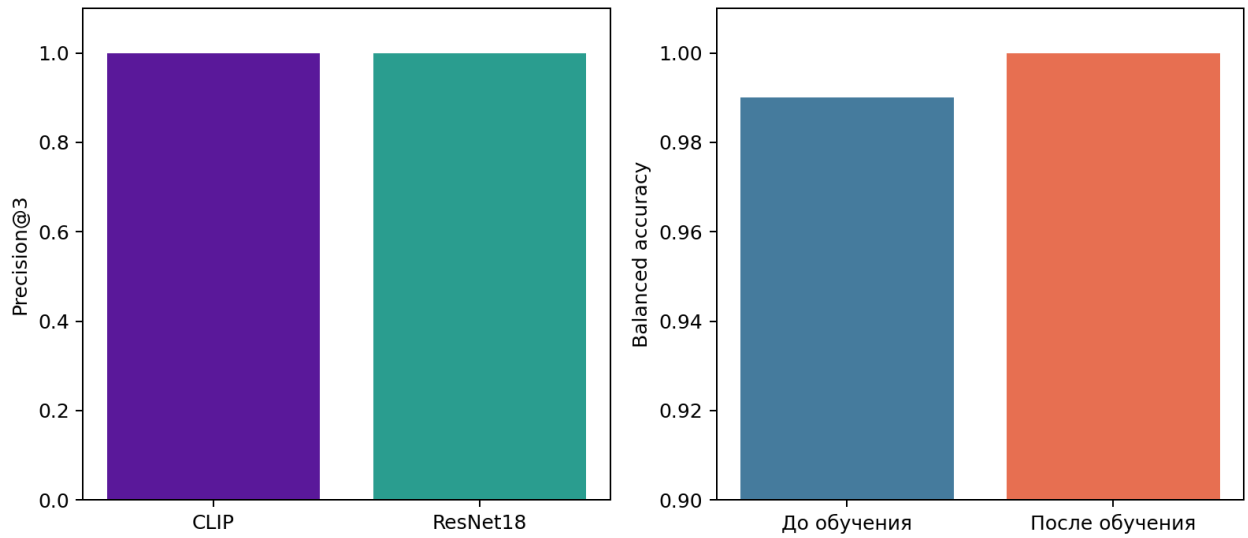


Рисунок 6 — Контроль точности выдачи поиска и метрики сходства.

5. Обучение метрики сходства

Для более узкой предметной области поверх визуальных признаков обучается небольшая проекция с triplet loss. Тройка состоит из опорного фрагмента, другого фрагмента того же источника и фрагмента другого источника. Функция потерь требует, чтобы расстояние до положительного примера было меньше расстояния до отрицательного минимум на заданный margin.

Алгоритм: 1) сформировать тройки только из обучающих источников; 2) получить базовые признаки; 3) обучить проекцию; 4) выбирать порог только по validation; 5) один раз оценить test; 6) сравнить balanced accuracy и precision@k до и после обучения.

Листинг 5.1 — один шаг обучения triplet loss

```
anchor = F.normalize(head(features_anchor), dim=1)
positive = F.normalize(head(features_positive), dim=1)
negative = F.normalize(head(features_negative), dim=1)
loss = F.triplet_margin_loss(anchor, positive, negative, margin=0.5)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Таблица 6

Показатель	До обучения	После обучения
Balanced accuracy	0,990	1,000
Размер выхода	базовый вектор	нормированный вектор проекции
Критерий	cosine similarity	cosine similarity после triplet loss

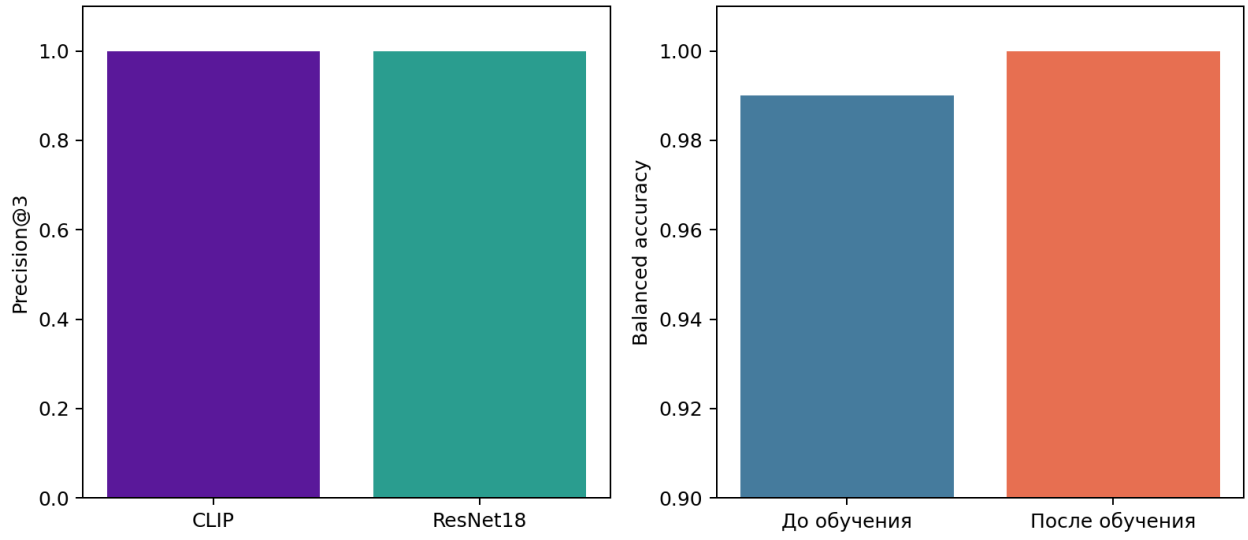


Рисунок 7 — Сравнение базовых признаков и признаков после обучения метрики.

6. Нейросети без дополнительного обучения (zero-shot)

Zero-shot-модели получают текстовые названия классов и применяются без дообучения на текущем наборе. Проверяются YOLO-World для открытого словаря детекции, CLIP для классификации и поиска, CLIPSeg для маски по тексту, а также комбинация Grounding DINO и SAM. Запросы формулируются по-английски, так как большинство моделей обучалось преимущественно на англоязычных подписях.

Алгоритм: 1) задать одинаковые текстовые запросы; 2) запустить модели на одном наборе изображений; 3) сохранить все ответы, включая пустые; 4) сравнить число объектов, маски и уверенности с цветовой маской; 5) не трактовать уверенность модели как подтверждение вида бактерии.

Листинг 6.1 — пример запуска YOLO-World и CLIPSeg

```
world = YOLO("yolov8s-world.pt")
world.set_classes(["Treponema pallidum", "spirochete", "bacterium"])
result = world("data/raw/treponema_primary.jpg", conf=0.10)[0]

inputs = clipseg_processor(text=["spiral bacterium"], images=[image],
                           return_tensors="pt")
with torch.no_grad():
    mask_logits = clipseg_model(**inputs).logits
mask = torch.sigmoid(mask_logits)[0].cpu().numpy() > 0.5
```

Таблица 7

Метод	Получено	Интерпретация
YOLO-World	0 устойчивых объектов	Слово не гарантирует визуальное распознавание
FastSAM text	1 область	Грубая область интереса
CLIP	относительная оценка 0,162	Полезно для ранжирования
CLIPSeg	0 устойчивых масок	Нужна специализированная сегментация
DINO + SAM	6 областей	Требуется визуальная проверка

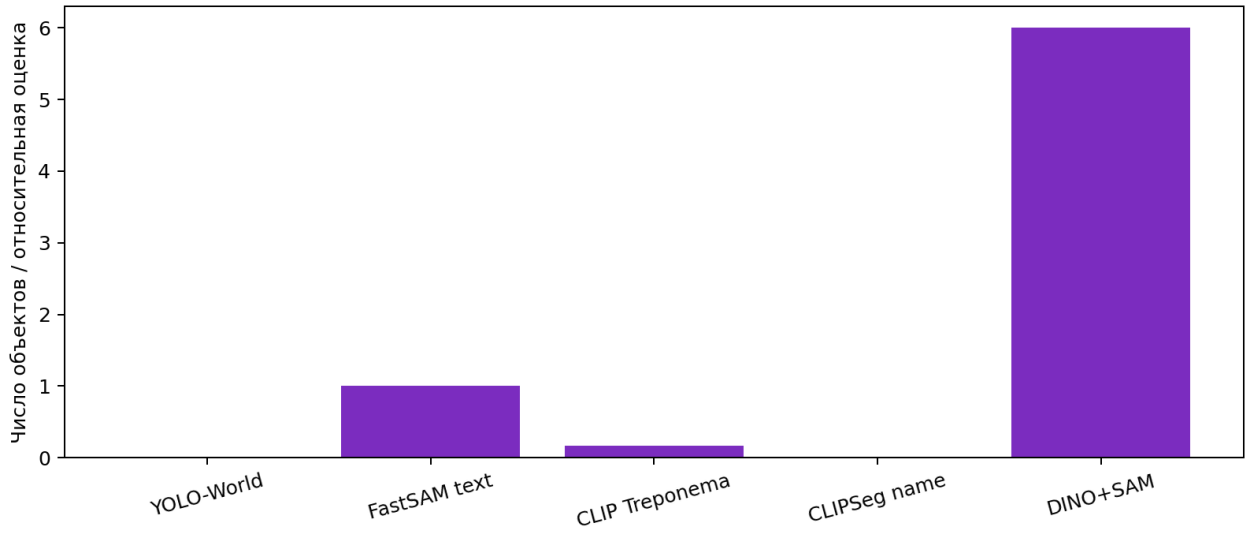


Рисунок 8 — Сводка ответов zero-shot-моделей на изображениях бледной трепонемы.

7. Поиск момента в видео и интерфейс Streamlit

Для локального поиска используется историческая запись Жана Командона с бледной трепонемой. Видео делится на кадры с шагом 2 с. Для каждого кадра вычисляется CLIP-вектор и сохраняется вместе с временем. Streamlit принимает текстовый запрос, кодирует его и показывает три кадра с наибольшим сходством.

Алгоритм: 1) открыть локальное видео; 2) определить FPS и число кадров; 3) с шагом 2 с сохранить кадры; 4) вычислить и нормировать CLIP-векторы; 5) записать index.json и vectors.npy; 6) в Streamlit получить запрос; 7) исключить почти соседние результаты; 8) показать кадр, время и оценку сходства.

Листинг 7.1 — индексирование видео и запуск Streamlit

```
# Из каталога lab4_treponema
python scripts/video_index.py data/raw/t07.ogv --out data/video_index --step 2
streamlit run scripts/app.py

# Основной фрагмент индексатора
for frame_number in range(0, frame_count, round(fps * step)):
    capture.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    ok, frame = capture.read()
    if not ok:
        continue
    vector = model.get_image_features(**processor(images=frame_rgb,
                                                  return_tensors="pt"))
    vectors.append(F.normalize(vector, dim=1)[0].numpy())
```

Таблица 8

Параметр	**Значение**
Видео	Ultramicroscope time-lapse of syphilis parasite, Jean Comandon
Источник	Wikimedia Commons, public domain
Разрешение / FPS / длительность	360 × 288 / 12 / 74,83 с
Шаг индекса	2 с
Ожидаемое число записей	около 38 кадров
Пример запроса	Treponema pallidum spirochetes moving in a microscope field

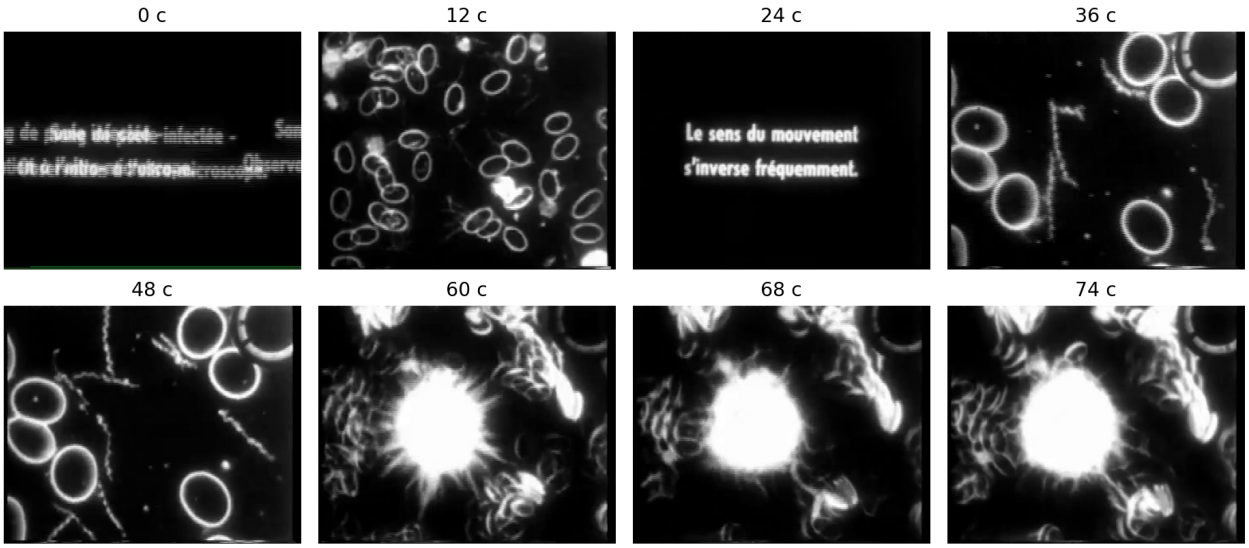


Рисунок 9 — Кадры нового видео, подготовленного для поиска в Streamlit.

7.1. Самостоятельный запуск с RuTube

RuTube в вычислительном эксперименте не запускался. Код выполняет проверку HTTPS-ссылки, загрузку выбранного публичного ролика через yt-dlp и построение того же локального индекса. При самостоятельном запуске подставляется ссылка вида https://rutube.ru/video/32-символьный-ID/, после чего в таблицу переносятся название ролика, число кадров, запрос, три времени и оценки сходства.

Листинг 7.2 — команды для самостоятельного запуска RuTube

```
python scripts/rutube_download.py   "https://rutube.ru/video/ВАШ_32-СИМВОЛЬНЫЙ_ID/"   --out
data/rutube --step 2
streamlit run scripts/app.py

# Если ролик уже скачан вручную:
python scripts/rutube_download.py   "https://rutube.ru/video/ВАШ_32-СИМВОЛЬНЫЙ_ID/"
--local-video "C:/Videos/video.mp4" --out data/rutube --step 2
```

Таблица 9

Поле для заполнения после запуска	**Значение**
Ссылка и название ролика	_____
Число проиндексированных кадров	_____
Текстовый запрос	_____
Три найденных времени	_____
Оценки сходства	_____

8. Автоматическая разметка и импорт в CVAT

Для CVAT подготавливаются изображения, прямоугольные рамки и три точки каждого видимого фрагмента. Рамка вычисляется по связной области маски, точки — по главной оси. При импорте нужно проверить каждый объект вручную: пересекающиеся спирали, обрезанные концы и цветные тени могут давать ошибочные компоненты.

Алгоритм: 1) построить маску; 2) найти крупные компоненты; 3) вычислить рамку и ключевые точки; 4) сохранить изображения задачи; 5) сформировать XML CVAT for images 1.1; 6) упаковать XML и изображения; 7) создать проект CVAT с меткой treponema; 8) импортировать архив и выполнить ручную проверку.

Листинг 8.1 — формирование рамки и точек для CVAT

```
x, y, width, height, area = stats[label]
box = {"xtl": x, "ytl": y, "xbr": x + width, "ybr": y + height}
points = [
    {"label": "start", "x": float(keypoints[0, 0]), "y": float(keypoints[0, 1])},
    {"label": "center", "x": float(keypoints[1, 0]), "y": float(keypoints[1, 1])},
    {"label": "end", "x": float(keypoints[2, 0]), "y": float(keypoints[2, 1])},
]
```

Таблица 10

Вход	Выход	Контроль
Основное изображение и цветовая маска	6 рамок и 18 точек; XML и ZIP для CVAT	Координаты внутри кадра; порядок start-center-end

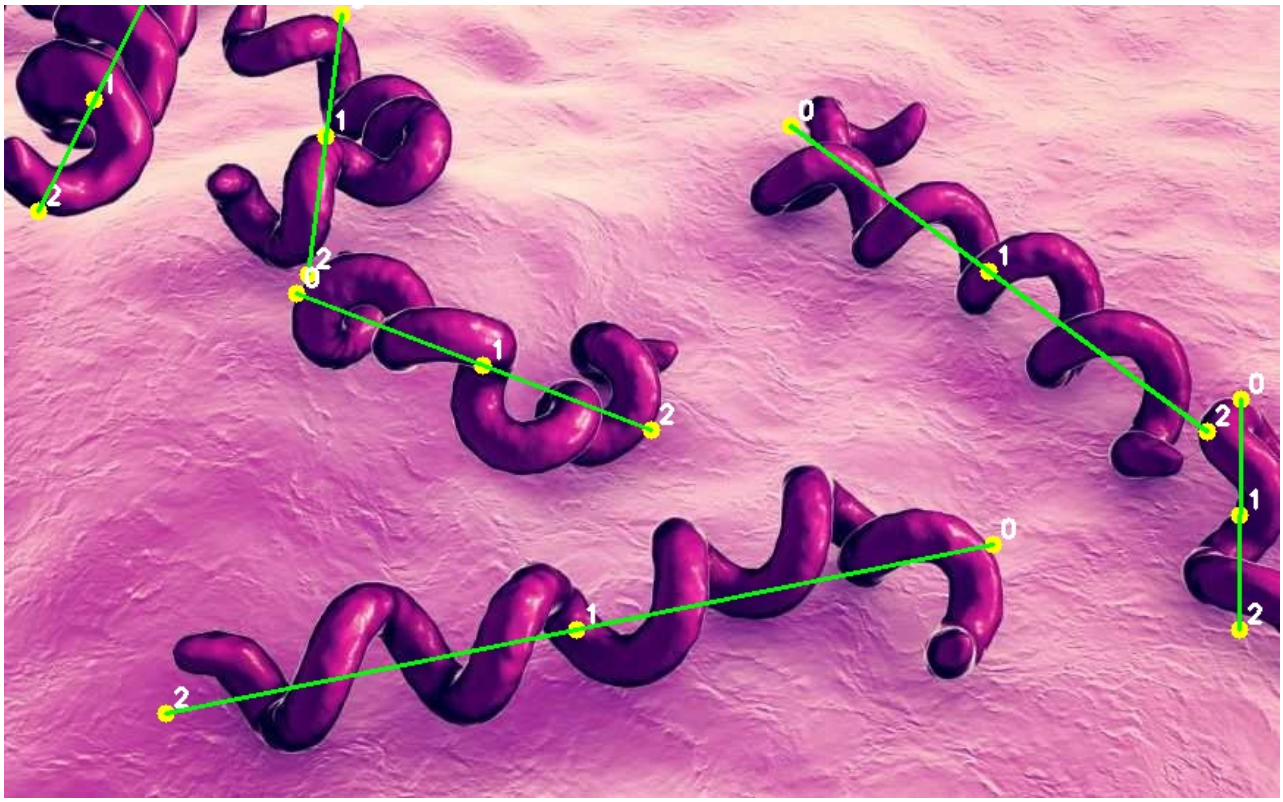


Рисунок 10 — Автоматические ключевые точки перед ручной проверкой в CVAT.

9. Учёт экспериментов в MLflow

MLflow хранит параметры запуска, метрики, изображения и модели. Для данной работы отдельные запуски создаются для поиска, triplet-проекции, трекеров и YOLO-pose. Это позволяет сравнивать методы по одним и тем же метрикам и повторно загрузить сохранённую модель.

Алгоритм: 1) создать локальное хранилище; 2) задать эксперимент; 3) в каждом запуске записать параметры; 4) после вычисления записать метрики; 5) приложить графики; 6) сохранить модель; 7) зарегистрировать лучшую версию и проверить повторную загрузку.

Листинг 9.1 — регистрация параметров и метрик

```
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("treponema_lab4")
with mlflow.start_run(run_name="tracker_csrt"):
    mlflow.log_params({"tracker": "CSRT", "video": "t07.ogv"})
    mlflow.log_metric("mean_iou", 0.488)
    mlflow.log_metric("fps", 62.7)
    mlflow.log_artifact("results/tracking_metrics.png")
```

Таблица 11

Группа запусков	Число вариантов	Основная метрика
Трекеры	4	IoU и FPS
Метрика сходства	2	Balanced accuracy
Классификаторы	2	Macro-F1
YOLO-pose	2	mAP50-95(P) и ошибка точки

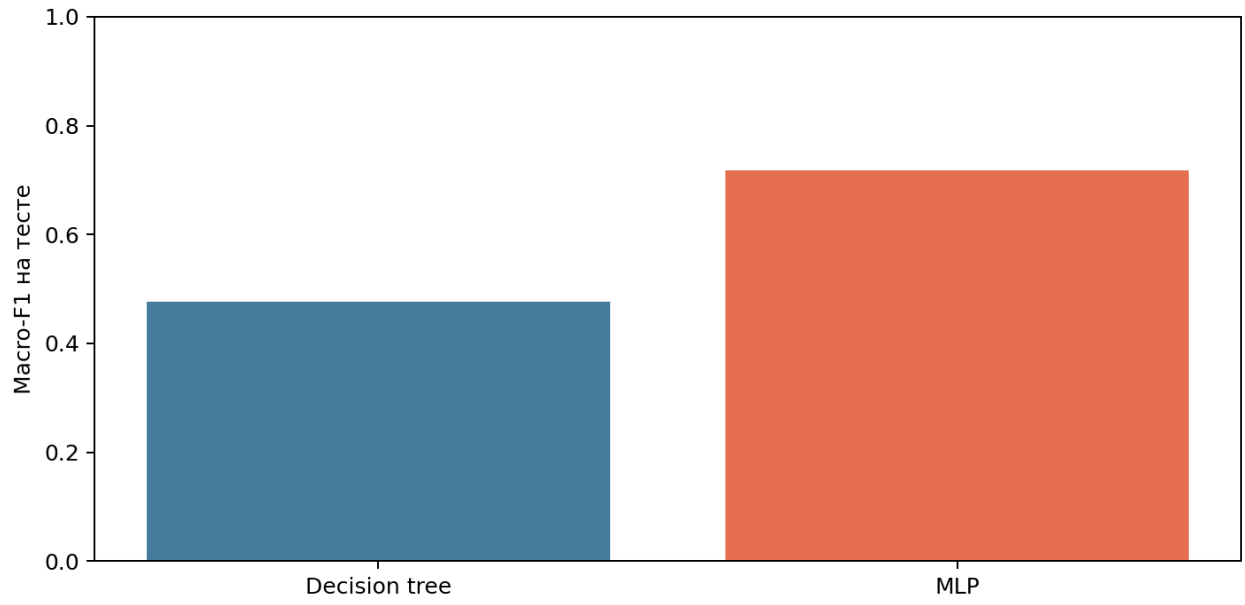


Рисунок 11 — Сравнение моделей, записанное как результат экспериментов MLflow.

10. Полный цикл YOLO-pose

Финальная часть объединяет подготовку разметки, обучение и проверку pose-модели. Для каждой трепонемы используются три точки. В data.yaml задаётся kpt_shape: [3, 3], то есть три точки с координатами x, y и видимостью. Горизонтальное отражение меняет местами начало и конец, поэтому flip_idx равен [2, 1, 0].

Алгоритм: 1) выгрузить разметку из CVAT; 2) разделить данные по исходным изображениям; 3) проверить формат YOLO-pose; 4) обучить две архитектуры с одинаковыми параметрами; 5) выполнить val на отложенных данных; 6) сравнить mAP и ошибку точки; 7) сохранить веса и пример предсказания.

Листинг 10.1 — обучение и сравнение pose-моделей

```
from ultralytics import YOLO

for weights, name in (("yolov8s-pose.pt", "pose_v8s"),
                      ("yolo11s-pose.pt", "pose_11s")):
    model = YOLO(weights)
    model.train(data="data/pose/dataset.yaml", epochs=30, imgsz=640,
                batch=8, project="results/pose", name=name, seed=0)
    metrics = YOLO(f"results/pose/{name}/weights/best.pt").val(
        data="data/pose/dataset.yaml", verbose=False)
    print(name, metrics.pose.map, metrics.box.map)
```

Таблица 12

Модель	mAP50-95(P)	Ошибка точки, px	PCK@0.1
YOLOv8s-pose	0,392	52,05	0,067
YOLO11s-pose	0,635	42,27	0,600

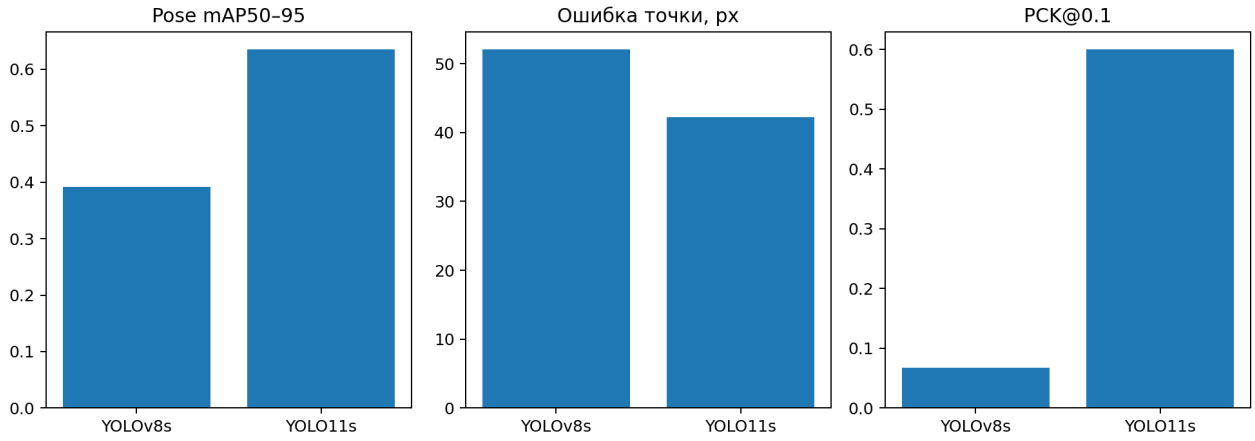


Рисунок 12 — Сравнение двух pose-моделей для схемы ключевых точек бледной трепонемы.

YOLO11s-pose показала лучший результат по всем трём показателям. Значения относятся к малому учебному набору и

используются для сравнения двух запусков в одинаковых условиях. Для устойчивой модели требуется расширить набор независимыми микрофотографиями, вручную проверить точки и сохранить отдельную тестовую выборку.

11. Заключение

В работе подготовлен единый набор материалов по Treponema pallidum: основная цветная иллюстрация, шесть дополнительных открытых изображений и историческое микроскопическое видео. На основном изображении выделено шесть крупных областей, построены маска, скелет и три ключевые точки каждого фрагмента. Для видео сравнены трекеры, подготовлены кадры и код поиска в Streamlit. Рассмотрены поиск похожих изображений, triplet loss, zero-shot-модели, CVAT, MLflow и полный цикл YOLO-pose.

Для RuTube предоставлен отдельный код, но сетевой запуск не выполнялся. После самостоятельного запуска в раздел 7.1 следует добавить ссылку, название ролика, число кадров, запрос, три найденных времени, оценки сходства и снимок окна Streamlit.

Список использованных источников

1. Методические указания по выполнению лабораторной работы № 4 по дисциплине «Компьютерное зрение». КНИТУ-КАИ, 2026.

2. PCR.News. Изображение Treponema pallidum. URL: https://pcr.news/upload/iblock/3a5/3a5ebdd65880badcc702e1bed5c07618.jpg

3. Wikimedia Commons. Treponema pallidum: открытые изображения и фильм Жана Командона. URL: https://commons.wikimedia.org/

4. OpenCV documentation: Image Processing and Tracking API. URL: https://docs.opencv.org/4.x/

5. Radford A. et al. Learning Transferable Visual Models From Natural Language Supervision. ICML, 2021.

6. Schroff F., Kalenichenko D., Philbin J. FaceNet: A Unified Embedding for Face Recognition and Clustering. CVPR, 2015.

7. Ultralytics documentation: Pose and YOLO-World. URL: https://docs.ultralytics.com/

8. CVAT documentation. URL: https://docs.cvat.ai/docs/

9. Zaharia M. et al. Accelerating the Machine Learning Lifecycle with MLflow. IEEE Data Engineering Bulletin, 2018.

10. Streamlit documentation. URL: https://docs.streamlit.io/